Assignment 3. Maze Escape

Course	Design and Analysis of Algorithms
Topics	Disjoint Set Union, minimum spanning trees (Kruskal, Prim), DFS, strongly connected components, topological ordering, shortest paths in DAGs
Weight	100 points: assignment 50, defense 50
Language	Java 17 or newer. No other language and no graph or DSU libraries.
Submission	Link to your GitHub repository, uploaded in Moodle before the deadline.

1. The task

A maze is a graph: a cell is a vertex, a wall between two neighbouring cells is an edge. You build a maze with Kruskal and your own DSU, find the way out with DFS, and rebuild the maze as the cheapest possible one with Kruskal and Prim.

The maze also has one-way slides between cells. You find their zones (strongly connected components) and use the order of the zones to find the cheapest route from the start to the exit.

Every student has personal files made from the student ID, so every number in your output is your own. The starter project has a file START_HERE.md: read it first, it explains the setup, what the numbers in the files mean, where each mission is written, how to check yourself and how to use git. Your program also writes a page output/maze.html: open it in a browser to see your maze, to walk through it with the arrow keys and to see the result of every mission on the picture.

2. Your input files

The starter project contains tools/MakeVariant.java. Run it with your student ID:

```
java tools/MakeVariant.java YOUR_ID data
```

It creates two files in the folder data.

walls.txt. The first line is "R C W": rows, columns, number of inner walls. Each of the next W lines is "a b w": a wall between the neighbouring cells a and b, and w, the cost of breaking it. All costs are different. Lines of walls are numbered from 1.

slides.txt. The first line is "n S": the number of cells and the number of slides. Each of the next S lines is "a b w": a one-way slide from cell a to cell b that takes w seconds. Slides ignore walls. There is no slide from a cell to itself, and the exit can always be reached from the start by slides.

Cells are numbered from 0, row by row: the cell in row r and column c has the number r*C + c. The start is cell 0 (top left), the exit is cell R*C - 1 (bottom right). A tiny example, 2 rows and 2 columns:

```
walls.txt     slides.txt     cells
2 2 4         4 4            0 1
0 1 17        0 1 3          2 3
0 2 5         1 0 2
1 3 9         1 2 4
2 3 23        2 3 5
```

3. Program and output

Every mission has its own file in src, Main.java is ready and prints the lines. Compile and run:

```
javac -d out src/*.java
java -cp out Main
```

Without arguments the program reads data/walls.txt and data/slides.txt. The command java tools/Check.java compiles the program and checks it on three examples with the right output.

The program prints the 13 lines below, each as "name: whole number", in this order. Other lines are allowed. The program must not wait for keyboard input.

Output line	Meaning	Mission
Broken	Number of walls broken when you take the walls in the order of the file (Mission 1).	1
Kept	Number of walls that stay.	1
Last broken wall	Line number in walls.txt of the last wall that was broken.	1
MST cost	Total cost of the walls broken in the cheapest maze.	2
Heaviest broken wall	The largest cost among the walls broken in the cheapest maze.	2
Path length	Number of moves from the start to the exit in the maze of Mission 1.	3
DFS visited	Number of cells DFS has marked at the moment it marks the exit; the start and the exit are counted. DFS tries the neighbours in the order up, right, down, left.	3
Dead ends	Number of cells with exactly one open door (the start and the exit count too).	3
Zones	Number of strongly connected components of the slides. A zone is a set of cells such that from any cell of it you can reach any other one by slides. A cell without a way back is a zone of its own.	4
Largest zone	The number of cells in the biggest zone.	4
Is DAG	1 if the slides have no cycle, otherwise 0. The slides have no cycle exactly when every cell is a zone of its own.	4
Flow stages	The graph of zones has one vertex per zone and an edge X to Y if some slide goes from a cell of X to a cell of Y. Flow stages is the number of zones on its longest chain.	5
Fastest route cost	In the graph of zones the edge X to Y costs the time of the cheapest slide from X to Y; walking inside a zone is free. The cost of the cheapest route from the zone of the start to the zone of the exit.	5

Output for the tiny files above, to check your reading of the rules:

```
Broken: 3   Kept: 1   Last broken wall: 3   MST cost: 31   Heaviest broken wall: 17
Path length: 2   DFS visited: 3   Dead ends: 2
Zones: 3   Largest zone: 2   Is DAG: 0   Flow stages: 3   Fastest route cost: 9
```

4. Missions (50 points)

Mission	What to do	Points
0. DSU	Class DisjointSet with find, union (returns false if the cells are already in one group), connected and count. Union by size and path compression. Your own checks added to DisjointSetTest.	5
1. Build the maze	Take the walls in the order of the file. Break a wall if union returns true; otherwise the wall stays. README: why Broken is always R*C - 1, and why Kept is always (R-1)*(C-1).	7
2. The cheapest maze	Break walls so that all cells are still connected and the total cost is the smallest. Implement Kruskal with your DSU and Prim with a priority queue; both must give the same MST cost.	8
3. Escape	DFS from the start to the exit in the maze of Mission 1. Think about a maze of 1000 by 1000 cells: the program must not overflow the stack. README: why there is exactly one path.	6
4. Zones	Find the zones of the slides with Kosaraju (two passes, DFS with a stack). The lists of slides for each cell are ready in Graphs. Number the zones in the order in which the second pass finds them: this is a topological order of the zones. README: why the second pass goes on the reversed graph, and why the slides have no cycle exactly when Zones equals the number of cells.	8
5. Fastest route	Build the graph of zones and go through the zones in the order of their numbers: compute Flow stages and the cheapest route to the exit. Do not use a priority queue here. README: why this order is enough, and the cost of the pass.	6
6. Git and README	Nothing to code. A commit after every mission with a message that says what was done (3 points). The questions in README answered in your own words and the section AI use filled in (2 points).	5
7. Picture, timing, README	Nothing to code. The ready class Render writes output/maze.png and the page output/maze.html (1 point: the picture is in the repository). Run the ready class Bench and put the times of Kruskal and Prim on 100x100, 300x300 and 1000x1000 mazes into README with two or three sentences on how the time grows (2 points). Fill in the table of time and space complexity of the algorithms you wrote (2 points).	5

Each mission uses the previous ones, so do them in order. The deadline is set in Moodle: upload the link to your repository there before it.

5. Submission

Submit the link to your GitHub repository in Moodle, for example https://github.com/username/maze-escape. The repository must be public.

- All source code is in the repository. Work on the main branch and commit after every mission with a message that says what was done (at least 7 commits). The main branch compiles.
- The report is the README.md of the repository: your ID, the output of your program, the answers, the AI use note, the timing table and the complexity table. The starter project already has the template.
- data/walls.txt, data/slides.txt and output/maze.png are in the repository.

A submission without a working link or without the README report is not accepted.

6. Defense (50 points)

The defense takes 5 minutes and is worth 50 points: one mini task and two questions. You come alone, with your laptop and your code open. Phones, messages and AI tools are not allowed during the defense.

7. Rules

- Your own work only. Copied code, a shared solution or unauthorized collaboration gives zero for the assignment. Code or text taken from a book or a website must be cited in the README with the source.
- AI tools (ChatGPT, Copilot and similar) may be used as a helper: to explain an idea, to understand an error, to review your code. The syllabus requires you to say so in writing: the README has a section "AI use" where you name the tool and what you used it for (write "none" if you did not use one). AI-generated work submitted without changes or without this note is academic dishonesty. You must understand every line you submit and be able to explain it yourself; if you cannot, the points for the assignment can be reduced.
- Deadlines. Submit the link in Moodle before the deadline: email and messengers are not accepted. A late submission or a late defense gets zero. An extension is possible only if you ask before the deadline and have a documented reason.
- Do not use ready-made graph or DSU libraries. The Java standard library (collections, priority queue) is allowed.